



— WEEK 2

AI Open Source Software

Plan, Track, and Visualize Your Work

Prof. Sejin Chun

Dong-A University



SECTION 01

작업 계획, 추적 및 시각화

Agile 방법론을 기반으로
GitHub의 도구를 활용하여
효율적인 협업 워크플로우 구축하기

- ✓ Agile & Scrum 기초
- ✓ GitHub Issues & Projects
- ✓ 시각화 및 자동화



학습 목표

이 세션을 마치면 다음 핵심 역량을 갖추게 됩니다



Agile 방법론 이해

소프트웨어 개발의 핵심 가치인 Agile 선언문과 12원칙을 이해하고, 변화에 유연하게 대응하는 마인드셋을 함양합니다.



GitHub 작업 관리

GitHub Issues와 Projects v2 기능을 활용하여 작업을 체계적으로 생성, 분류하고 추적하는 방법을 익힙니다.



작업 시각화 기법

Burndown Chart, Burnup Chart, CFD 등 다양한 차트를 통해 프로젝트 진행 상황을 한눈에 파악하고 병목을 발견합니다.



팀 생산성 향상

명확한 역할 분담과 효율적인 커뮤니케이션을 위한 워크플로우를 구축하고, Sprint Planning을 직접 실습합니다.



SECTION 02

왜 계획과 추적이 중요한가?

개발 프로세스의 불확실성을 줄이고
팀의 예측 가능성을 극대화하는
핵심 전략과 필요성

- 🎯 목표 정렬 및 진행 투명성 확보
- 🔍 병목 현상 및 리스크 조기 발견
- 👥 효율적인 리소스 배분과 협업



계획 없는 개발의 문제점

명확한 계획과 추적 프로세스가 부재할 때 발생하는 5가지 주요 리스크



목표 불명확

프로젝트의 최종 목적지가 모호하여 팀원들이 서로 다른 방향으로 개발을 진행하게 됩니다.



우선순위 혼란

무엇이 중요한지 합의되지 않아, 가치가 낮은 작업에 불필요한 시간을 낭비하게 됩니다.



진행 상황 파악 불가

현재 어디까지 왔는지, 언제 완료될지 예측할 수 없어 일정 관리가 불가능해집니다.



병목 지점 미발견

특정 구간에서 작업이 적체되어도 이를 늦게 발견하여 전체 프로젝트가 지연됩니다.



팀 협업 어려움

누가 무엇을 하는지 알 수 없어 업무 중복이나 누락이 발생하고 커뮤니케이션 비용이 증가합니다.



체계적인 계획의 이점

개발 프로세스에 계획을 도입함으로써 얻을 수 있는 핵심 가치



명확한 목표와 방향

팀 전체가 동일한 목표를 공유하며, 불필요한 혼선을 줄이고 개발 방향성을 명확히 유지합니다.



투명한 진행 상황

누구나 프로젝트의 현재 상태를 쉽게 파악할 수 있어, 이해관계자 간의 신뢰도를 높입니다.



효율적인 리소스 배분

팀원의 역량과 시간을 최적의 작업에 배치하여 리소스 낭비를 최소화하고 생산성을 극대화합니다.



빠른 문제 발견 및 해결

잠재적인 위험 요소와 병목 지점을 조기에 식별하여, 큰 문제로 발전하기 전에 신속하게 대응합니다.



팀 동기부여 향상

작은 단위의 성취를 지속적으로 경험하게 하여, 팀원들의 사기를 높이고 자율적인 업무 환경을 조성합니다.



SECTION 02

Agile 기본 개념

소프트웨어 개발의 패러다임 전환.
변화하는 요구사항에 신속하게 대응하고
지속적으로 가치를 전달하는 핵심 원칙

 Agile 선언문 핵심 가치

 Agile 12원칙

 고객 협력과 변화 수용



| Agile 선언문 핵심 가치

소프트웨어 개발의 더 나은 방법 (Manifesto for Agile Software Development)



프로세스와 도구보다 >

개인과 상호작용을

01



포괄적인 문서보다 >

작동하는 소프트웨어를

02



계약 협상보다 >

고객과의 협력을

03



계획을 따르기보다 >

변화에 대응하기를

04



Agile 12원칙 (요약)

Agile 선언문을 뒷받침하는 12가지 실행 지침

고객 만족

가치 있는 소프트웨어를 지속적으로 전달하여 고객을 만족시킵니다.

변화 수용

개발 후반부라도 요구사항 변경을 환영하며 경쟁력을 높입니다.

자주 전달

작동하는 소프트웨어를 몇 주에서 몇 달 단위로 자주 전달합니다.

협업

비즈니스 담당자와 개발자는 매일 함께 일하며 소통합니다.

동기부여

동기 부여된 개인을 중심으로 팀을 구성하고 환경을 지원합니다.

대면 소통

가장 효율적이고 효과적인 정보 전달 방법은 얼굴을 보고 대화하는 것입니다.

작동하는 SW

작동하는 소프트웨어는 진척상황을 측정하는 가장 주된 척도입니다.

지속 가능성

일정한 개발 속도를 무한히 유지할 수 있는 환경을 만듭니다.

기술적 우수성

기술적 우수성과 좋은 설계에 대한 지속적인 관심은 민첩성을 높입니다.

단순성

하지 않아도 되는 일의 양을 최대화하는 기술이 필수적입니다.

자기 조직화

최고의 아키텍처와 설계는 자기 조직화된 팀에서 나옵니다.

지속적 개선

팀은 정기적으로 더 효과적인 방법을 찾아 조율하고 행동을 조정합니다.



SECTION 03

Scrum 프레임워크

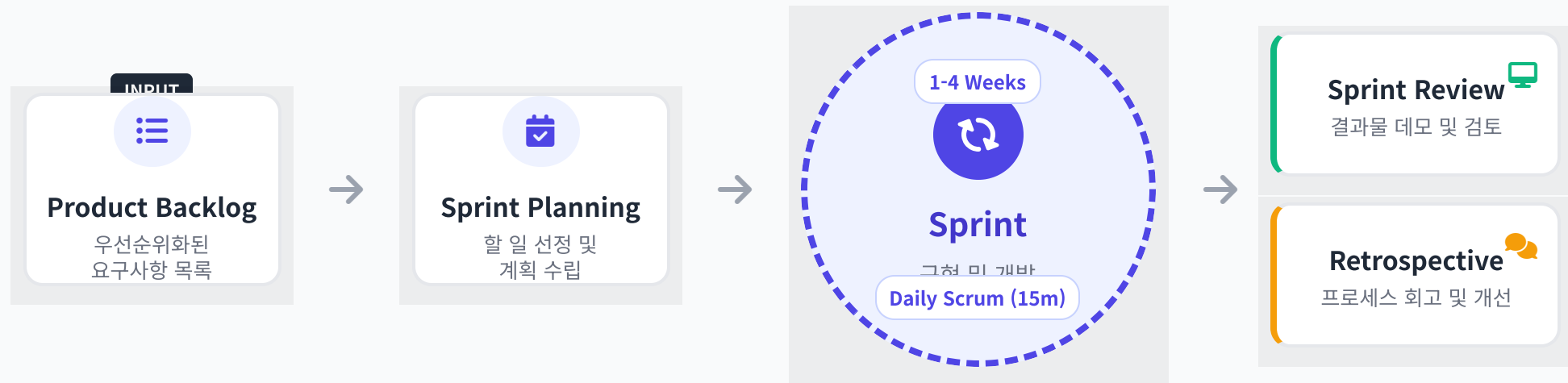
복잡한 적응형 문제에 대응하며
가치 있는 제품을 생산하기 위한
경량 애자일(Agile) 프레임워크

-  Roles (역할: PO, SM, Devs)
-  Events (이벤트: Sprint, Daily...)
-  Artifacts (산출물: Backlog...)



Scrum 프레임워크

반복적이고 점진적인 소프트웨어 개발 프로세스의 흐름





| Sprint 크기 권장사항

팀의 성숙도와 프로젝트 성격에 따른 Sprint 기간 선택 가이드



1주 Sprint

매우 빠른 피드백 주기를 가지며, 시장 변화에 즉각적으로 대응해야 하는 스타트업이나 초기 프로젝트에 적합합니다.

- ✓ 가장 빠른 피드백
- ✓ 방향 전환 용이
- ✗ 계획/회의 오버헤드 높음

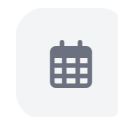


2주 Sprint

가장 일반적이고 권장되는 기간입니다. 작업 수행과 회의 사이의 이상적인 균형을 제공하며 지속 가능한 속도를 유지합니다.

- ✓ 이상적인 균형
- ✓ 충분한 작업 시간 확보
- ✓ 예측 가능성 높음

BEST CHOICE ★



3-4주 Sprint

복잡도가 매우 높거나 레거시 시스템, 하드웨어 연동 등 배포 비용이 큰 프로젝트에서 안정적인 개발을 위해 선택합니다.

- ✓ 안정적인 호흡
- ✗ 피드백 지연 위험
- ✗ 변화 대응 어려움



SECTION 04

GitHub Issues: 작업 관리의 시작

프로젝트의 모든 작업을 추적하고
팀원과 소통하는 협업의 출발점
버그부터 기능 요청까지

- ✓ Issue의 정의와 역할
- ✓ 효과적인 Issue 작성법
- ✓ 템플릿과 레이블 시스템



| Issue란?

GitHub 프로젝트에서 모든 작업과 커뮤니케이션이 시작되는 기본 단위



버그 및 오류

소프트웨어 동작 중 발생하는 예기치 않은 오류나 결함을 보고하고 수정 과정을 추적합니다.



새로운 기능 요청

프로젝트에 추가되었으면 하는 새로운 기능이나 아이디어를 제안하고 논의합니다.



문서 개선

README, 위키, 가이드 등 프로젝트 문서의 오류 수정이나 내용 보강 작업을 관리합니다.



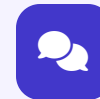
질문 및 토론

코드 구현 방법, 아키텍처 설계 등 프로젝트와 관련된 질문을 올리고 팀원들과 토론합니다.



기술 부채

당장은 동작하지만 장기적으로 개선이 필요한 코드 리팩토링이나 구조 개선 작업을 기록합니다.



협업의 중심

단순한 할 일 목록을 넘어, 코드 변경사항(PR)과 연동되어 작업의 문맥과 히스토리를 보존합니다.



효과적인 Issue 작성법: 좋은 제목

명확하고 검색 가능한 제목은 빠른 문제 해결의 첫걸음입니다

✖ 나쁜 예 (Bad Examples)

"버그"

! 너무 포괄적임. 어떤 버그인지 전혀 알 수 없음.

"로그인 안됨"

! 증상만 있고 상황(Context)이나 구체적인 조건이 누락됨.

"도와주세요!!!!"

! 감정적 호소는 정보 전달에 도움이 되지 않으며 검색이 불가능함.

✔ 좋은 예 (Good Examples)

[Bug] 로그인 시 이메일 형식 검증 실패

i 태그([Bug])로 분류하고, 구체적인 기능과 오류 내용을 명시함.

[Feature] 다크 모드 테마 지원 추가

i 목적(기능 추가)이 명확하며, 구현해야 할 대상이 구체적임.

[Docs] README.md 설치 가이드 오타 수정

i 수정 대상 파일과 작업 내용이 한눈에 파악됨.



Issue 템플릿 예시

체계적인 버그 리포트와 기능 요청을 위한 Markdown 템플릿 구조

 bug_report.md

BUG

```
---
name: Bug Report
about: 버그 리포트 작성
title: '[Bug] '
labels: bug
assignees: ''
---
```

🐛 버그 설명

버그에 대한 명확하고 간결한 설명

📋 재현 단계

1. '...'로 이동
2. '...'를 클릭
3. '...'까지 스크롤
4. 오류 발생

✅ 예상 동작

어떻게 동작해야 하는지 설명

❌ 실제 동작

 feature_request.md

FEATURE

```
---
name: Feature Request
about: 새로운 기능 제안
title: '[Feature] '
labels: enhancement
assignees: ''
---
```

🎯 해결하려는 문제

이 기능이 어떤 문제를 해결하나요?

<!-- 구체적인 문제 상황을 기술해주세요 -->

💡 제안하는 솔루션

어떻게 구현되면 좋을까요?

<!-- 원하는 기능의 상세 내용을 설명해주세요 -->

🔄 대안

고려한 다른 방법이 있나요?

<!-- 선택하지 않은 다른 해결책이 있다면 적어주세요 -->

📁 추가 컨텍스트



| Issue Template 설정

프로젝트 초기 설정 시 템플릿 파일들을 자동으로 생성하는 Bash 스크립트

>_ setup_templates.sh

BASH SCRIPT

```
# 1. .github/ISSUE_TEMPLATE/ 디렉토리 생성
mkdir -p .github/ISSUE_TEMPLATE

# 2. Bug report 템플릿 생성
cat > .github/ISSUE_TEMPLATE/bug_report.md << 'EOF'
---
name: Bug Report
about: 버그를 리포트합니다
title: '[Bug] '
labels: 'bug'
assignees: ''
---

## 🐛 버그 설명
(여기에 버그 설명을 작성하세요)

## 📋 재현 단계
1. ...
2.
```



SECTION 05

Labels: 이슈 분류하기

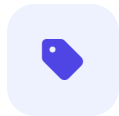
체계적인 레이블링 전략으로
이슈의 우선순위와 상태를
명확하게 관리하는 방법

- ✓ 기본 Label 체계 (Type, Priority)
- ✓ 상태 및 크기 관리 (Status, Size)
- ✓ GitHub Actions 자동화



| 기본 Label 체계

체계적인 이슈 관리를 위해 권장되는 4가지 핵심 분류 기준입니다.



타입 (Type)

이슈의 성격을 정의합니다. 버그 수정인지, 새로운 기능 추가인지, 문서 작업인지 등을 명확히 구분하여 작업의 종류를 파악합니다.

bug

enhancement

documentation



우선순위 (Priority)

작업의 긴급도와 중요도를 나타냅니다. 제한된 리소스 내에서 어떤 작업을 먼저 처리해야 할지 결정하는 기준이 됩니다.

critical

high

medium



상태 (Status)

작업의 현재 진행 단계를 추적합니다. 시작 전, 진행 중, 리뷰 대기, 완료 등의 상태를 통해 워크플로우를 관리합니다.

new

ready

blocked



크기 (Size)

작업의 예상 소요 시간이나 복잡도를 표시합니다. 티셔츠 사이즈(XS~XL) 등을 활용하여 작업량을 대략적으로 산정합니다.

size: XS

size: M

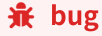
size: XL



Labels: 이슈 분류 체계

이슈의 성격(Type)과 긴급도(Priority)를 구분하는 표준 레이블 가이드

타입 (Type)

Label	Description
 bug	예상치 못한 오류 및 버그 수정
 enhancement	새로운 기능 추가 및 개선 사항
 documentation	README, Wiki 등 문서 관련 작업
 question	추가 정보가 필요하거나 논의가 필요한 질문
 maintenance	코드 리팩토링, 의존성 업데이트 등 유지보수

우선순위 (Priority)

Label	Description
 critical	즉시 해결 필요 (서비스 중단, 치명적 오류)
 high	높은 우선순위, 다음 배포 전 반드시 해결
 medium	중간 우선순위, 일반적인 개발 흐름에서 처리
 low	낮은 우선순위, 여유가 있을 때 처리



Label 분류 체계: 상태 및 크기

이슈의 진행 상황(Status)과 작업 규모(Size)를 명확히 정의하여 관리 효율성 증대

Status (상태)

LABEL	의미
 status: new	새로 등록된 이슈, 검토 전
 status: investigating	원인 파악 및 분석 중
 status: in-progress	현재 작업 진행 중
 status: ready	작업 완료, 리뷰 대기
 status: blocked	외부 요인으로 진행 불가

Size (크기)

LABEL	예상 소요 시간
 XS	1시간 미만 (간단 수정)
 S	반나절 (단일 작업)
 M	1 - 2일 (보통 기능)
 L	3 - 5일 (복잡 기능)
 XL	1주 이상 (분할 필요)



Label 자동화 설정

파일 변경 경로에 따라 자동으로 레이블을 부여하는 labeler.yml 작성

.github/labeler.yml

YAML

.github/labeler.yml 설정 파일

documentation:

- docs/**/*
- '**/*.md'

frontend:

- 'src/components/**/*'
- 'src/pages/**/*'

backend:

- 'src/api/**/*'
- 'src/services/**/*'

tests:

- '**/*.test.js'
- '**/*.spec.js'

설정 가이드

GUIDE



Label 이름 매핑

YAML 파일의 최상위 키(Key)가 자동으로 GitHub Label 이름이 됩니다.
(예: frontend, backend)



Glob 패턴 활용

**/* 패턴을 사용하여 하위 디렉토리의 모든 변경사항을 감지합니다. 특정 확장자만 타겟팅할 수도 있습니다.



자동 분류 원리

Pull Request가 생성되거나 업데이트될 때, 변경된 파일 목록을 스캔하여 일치하는 규칙의 레이블을 자동으로 부착합니다.



주의사항

해당 Label이 저장소에 미리 생성되어 있어야 오류 없이 동작합니다. 파일 위치는 반드시 .github/labeler.yml 이어야 합니다.



Label 자동화 Workflow

GitHub Actions를 활용하여 PR 생성 시 자동으로 레이블을 부착하는 설정

 .github/workflows/labeler.yml

YAML

```
name: Labeler
on: # Trigger condition
  - pull_request_target

jobs:
  triage:
    permissions:
      contents: read
      pull-requests: write
    runs-on: ubuntu-latest

    steps:
      - uses: actions/labeler@v4
        with:
          repo-token: "${{ secrets.GITHUB_TOKEN }}"
          sync-labels: true
```

 설정 포인트

INFO

on: pull_request_target

Fork된 저장소의 PR에서도 안전하게 동작하기 위해 `pull\_request` 대신 사용합니다. 원본 저장소의 권한 컨텍스트에서 실행됩니다.

permissions 설정

GitHub Actions 봇이 PR에 레이블을 추가하기 위해 `pull-requests: write` 권한이 명시적으로 필요합니다.

actions/labeler@v4

파일 변경 경로를 기반으로 레이블을 지정하는 공식 액션입니다. `.github/labeler.yml` 설정 파일을 참조합니다.



SECTION 07

GitHub Projects: 칸반 보드

강력한 프로젝트 관리 도구인
GitHub Projects v2를 활용하여
작업 흐름을 시각화하고 자동화하기

- 📅 Projects v2 (Table/Board/Roadmap)
- ☰ 칸반 보드 구성 및 Column 의미
- 📅 Milestones 및 워크플로우 자동화



GitHub Projects v2 (New Projects)

강력한 기능을 갖춘 현대적인 프로젝트 관리 도구의 핵심 기능과 템플릿



Multiple Views

스프레드시트 같은 Table, 칸반 스타일의 Board, 타임라인을 보여주는 Roadmap 등 다양한 뷰를 제공하여 프로젝트 데이터를 유연하게 시각화합니다.



Custom Fields

텍스트, 숫자, 날짜, 단일 선택(Single Select), 반복(Iteration) 등 프로젝트 특성에 맞는 메타데이터 필드를 자유롭게 정의하여 관리할 수 있습니다.



Automation & Insights

이슈 상태 변경 시 자동 이동과 같은 Built-in 워크플로우를 설정하고, 과거 데이터를 기반으로 한 차트를 통해 진행 상황을 분석합니다.



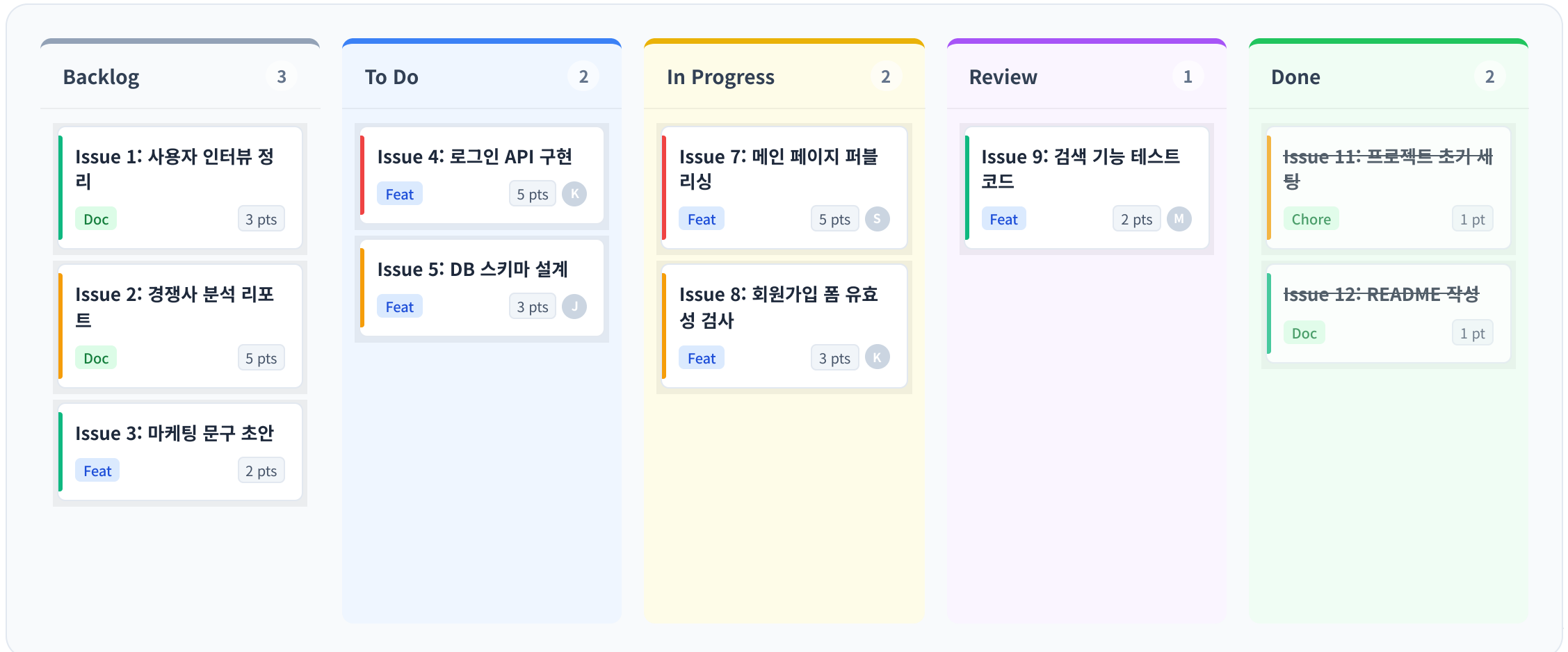
Quick Start Templates

Team backlog, Feature planning, Bug tracking 등 미리 정의된 템플릿을 선택하여 프로젝트 보드를 즉시 생성하고 시작할 수 있습니다.



| 칸반 보드 구성

작업의 흐름을 시각화하고 진행 상태를 한눈에 파악하는 보드 레이아웃 예시





| 칸반 보드 Column 의미

각 단계별 역할 정의 및 WIP(Work In Progress) 제한 권장사항

Column	의미 및 역할	WIP Limit
 Backlog	우선순위가 정렬된 전체 작업 목록	∞
 To Do	현재 Sprint에서 처리하기로 선정된 작업	∞
 In Progress	현재 개발자가 작업 중인 상태 (집중 유도)	3개
 Review	Pull Request 생성 후 코드 리뷰 대기	2개
 Done	테스트 및 배포가 완료되어 종료된 작업	∞



SECTION 08

Milestones: 목표 설정

일정 기간의 목표를 그룹화하고
프로젝트의 진행 단계를
체계적으로 관리하는 방법

- ✓ Milestone의 개념과 목적
- ✓ MVP와 버전 관리
- ✓ 효과적인 마일스톤 활용 팁



Milestone 예시

v1.0.0 MVP 출시를 위한 목표 설정과 진행 상황 추적

🚩 milestone_plan.md

PLAN

v1.0.0 - MVP 출시

📅 Due date: 2026-03-31

🎯 Goal: 초기 사용자 피드백을 위한 최소 기능

📋 주요 목표 (Objectives)

1. 사용자 인증 시스템 구축
2. 게시글 기본 CRUD 기능
3. 반응형 웹 UI 구현
4. AWS 배포 파이프라인 구성

💡 상세 설명

이 마일스톤은 서비스의 핵심 가치를 전달하는 최소한의 기능을 포함합니다. 부가적인 기능(예: 소셜 로그인, 알림)은 v1.1.0으로 연기합니다.

🔄 Deliverables

- 실행 가능한 웹 애플리케이션 URL
- API 명세서 (Swagger)
- 사용자 매뉴얼 v1.0

📊 milestone_status.md

TRACKING

📊 진행 상황 (Current Status)

Progress: 45% (9/20 issues)

✅ 완료된 작업 (Done)

- [x] #1 사용자 회원가입 API
- [x] #2 로그인 페이지 퍼블리싱
- [x] #5 데이터베이스 스키마 설계
- [x] #7 프로젝트 초기 설정 (React+Node)

🔄 진행 중 (In Progress)

- [] #3 JWT 인증 미들웨어 구현
- [] #4 메인 대시보드 UI 작업

📅 예정 (To Do)

- [] #8 CI/CD 스크립트 작성
- [] #9 프로필 수정 기능
- [] #12 통합 테스트 작성



Milestone 활용 팁

성공적인 프로젝트 관리를 위한 마일스톤 운영의 핵심 전략



명확한 목표 설정

"기능 개선"과 같은 모호한 목표 대신, "회원가입 프로세스 완료율 20% 향상"과 같이 구체적이고 측정 가능한 목표를 설정합니다.



현실적인 기한 준수

팀의 실제 가용 리소스와 예상치 못한 이슈를 고려하여 달성 가능한 현실적인 일정을 수립합니다. 무리한 일정은 품질 저하로 이어집니다.



정기적 검토 및 회고

주간 또는 격주 단위로 마일스톤 진행 상황을 리뷰하고, 지연되는 항목에 대해 조기에 대응책을 마련하는 습관을 들입니다.



유연한 조정

계획은 언제든지 변경될 수 있음을 인지하고, 비즈니스 요구사항이나 기술적 이슈에 따라 마일스톤의 범위나 일정을 유연하게 조정합니다.



SECTION 09

작업 시각화 기법

프로젝트의 진행 상황을 투명하게 공유하고
잠재적인 문제를 조기에 발견하기 위한
데이터 기반의 시각화 도구 활용

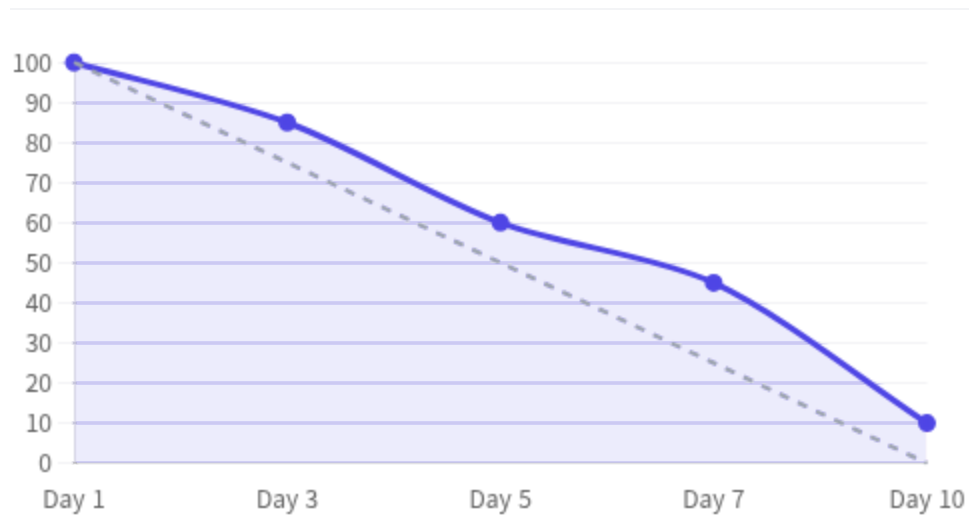
-  Burndown Chart (번다운 차트)
-  Burnup Chart (번업 차트)
-  Cumulative Flow Diagram (CFD)



Burndown vs Burnup Chart

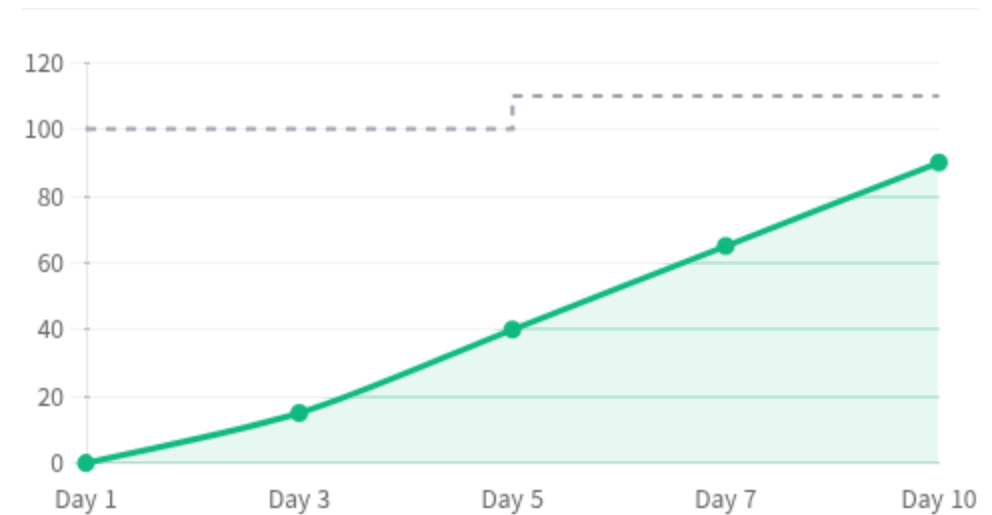
프로젝트 진행 상황과 범위를 시각화하는 두 가지 핵심 방법

 Burndown Chart



- **Ideal Line:** 이상적인 남은 작업량 (직선 감소)
- **Actual Line:** 실제 남은 작업량 (위=지연, 아래=빠름)

 Burnup Chart



- **Total Scope:** 전체 작업 범위 (변경 추적 가능)
- **Completed:** 완료된 작업량 (누적 증가)



Cumulative Flow Diagram (CFD)

작업의 상태별 누적 흐름을 통해 병목 현상과 진행 속도를 시각화

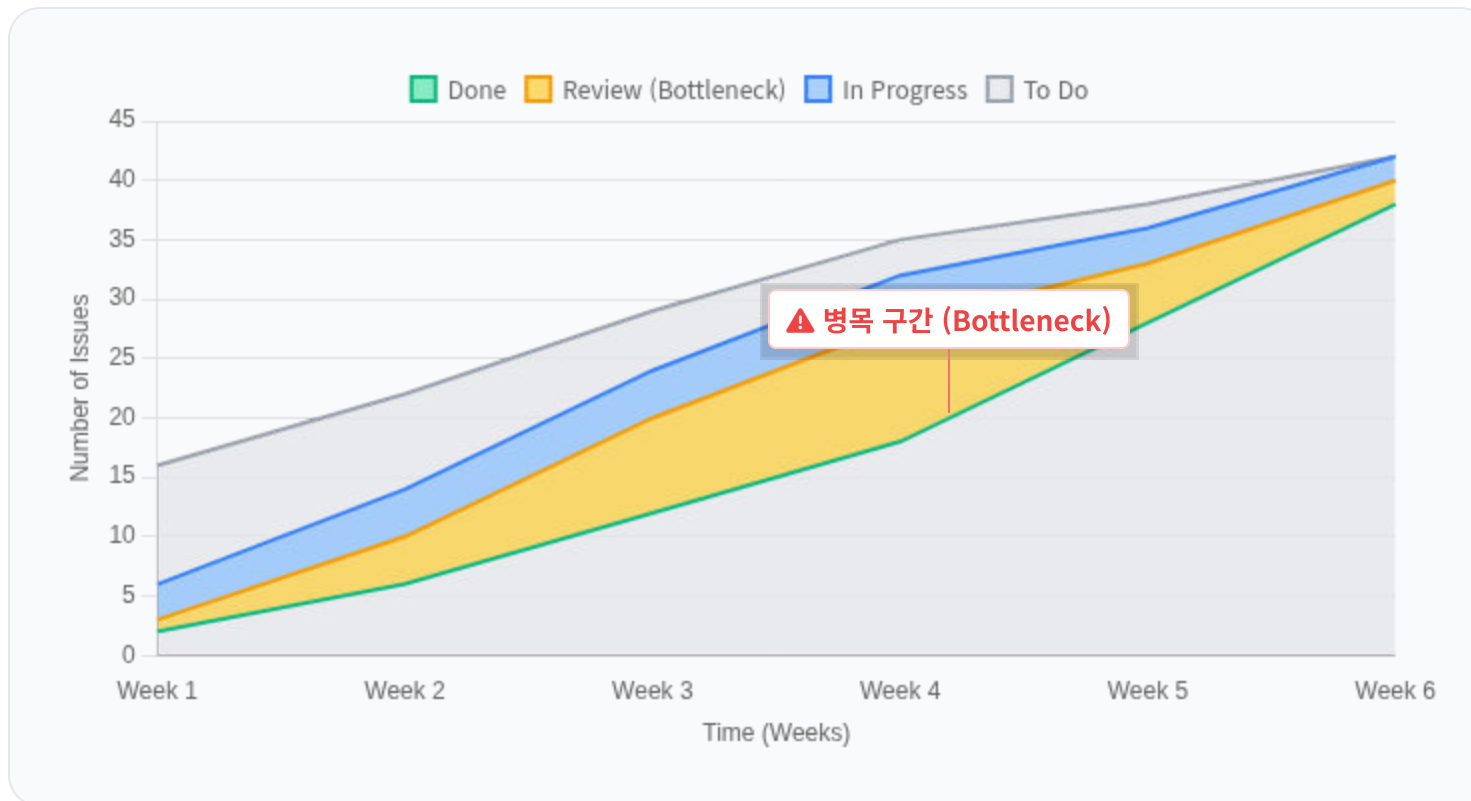


차트 해석 방법

가로축은 시간, 세로축은 작업의 누적 개수를 나타냅니다. 각 영역의 너비(수직 높이)는 해당 단계에 머물러 있는 작업(WIP)의 양을 의미합니다.

병목 발견

특정 상태의 영역이 급격히 넓어지면 해당 단계의 처리 속도가 느려져 작업이 쌓이고 있다는 뜻입니다. (예: Review 단계의 비대화)

리드 타임 예측

그래프의 수평 너비는 작업이 시작되어 완료되기까지 걸리는 평균 시간(Lead Time)을 나타냅니다.



SECTION 10

GitHub Projects 고급 기능

단순한 칸반 보드를 넘어,
워크플로우 자동화와 커스텀 필드로
프로젝트 관리 효율 극대화

- ✓ Custom Fields (커스텀 필드)
- ✓ Automation Rules (자동화)
- ✓ GitHub Actions Integration



Custom Fields

프로젝트 요구사항에 최적화된 메타데이터 관리 필드 5가지



Priority

SINGLE SELECT

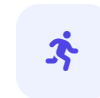
작업의 긴급성과 중요도를 High, Medium, Low 등으로 분류하여 처리 우선순위를 명확히 시각화합니다.



Story Points

NUMBER

피보나치 수열(1, 2, 3, 5, 8...) 등을 사용하여 작업의 복잡도와 필요한 노력을 정량적으로 추정합니다.



Sprint

ITERATION

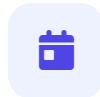
작업을 수행할 특정 기간(스프린트)에 할당하고, 팀의 주기적인 계획과 성과(Velocity)를 추적합니다.



Epic

TEXT / SELECT

여러 개의 관련 스토리와 작업을 포괄하는 큰 단위의 업무를 정의하여 프로젝트를 계층적으로 구조화합니다.



Due Date

DATE

작업의 최종 마감 기한을 설정하고, 타임라인(Roadmap) 뷰에서 일정 지연 여부를 시각적으로 확인합니다.



Automation 설정

GitHub Projects 워크플로우 자동화를 통한 효율적인 상태 관리 예시

New Issue Automation

WHEN Issue is **opened**

THEN Add to project & Set status to **To Do**

PR In-Progress

WHEN Pull request is **opened**

THEN Set status to **In Progress**

PR Merged Completion

WHEN Pull request is **merged**

THEN Set status to **Done**

Issue Completion

WHEN Issue is **closed**

THEN Set status to **Done**



GitHub Actions 통합

Project Automation Workflow 설정 예시 (.yaml)



.github/workflows/project-automation.yml

WORKFLOW

```
name: Project Automation

on:
  issues:
    types: [opened, closed]
  pull_request:
    types: [opened, closed]

jobs:
  add-to-project:
    runs-on: ubuntu-latest
    steps:
      - name: Add to project
        uses: actions/add-to-project@v0.4.0
        with:
          project-url: https://github.com/users/USERNAME/projects/1
          github-token: ${{ secrets.PROJECT_TOKEN }}
```



SECTION 09

Insights 활용

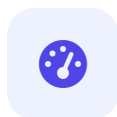
데이터 기반의 프로젝트 관리와
팀 생산성 향상을 위한
주요 메트릭 분석 방법

- ✓ Velocity & Cycle Time
- ✓ Throughput & WIP
- ✓ 커스텀 차트 생성



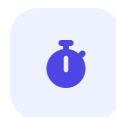
GitHub Insights 활용

데이터 기반의 프로젝트 관리와 팀 생산성 분석을 위한 4가지 핵심 지표



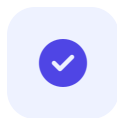
Velocity (속도)

Sprint별 완료된 Story Points의 합계입니다. 팀의 평균 처리 속도를 계산하여 향후 Sprint의 예측 가능성을 높이고, 일정한 개발 리듬을 유지하는 데 활용됩니다.



Cycle Time (사이클 타임)

작업(Issue)이 시작된 시점부터 완료될 때까지 걸린 시간입니다. 프로세스 내의 병목 지점을 파악하고 작업의 효율성을 진단하는 가장 중요한 척도 중 하나입니다.



Throughput (처리량)

특정 기간(주/월) 동안 완료된 Issue의 총 개수입니다. 팀의 전반적인 생산성 트렌드를 확인하고, 시간 경과에 따른 성과 변화를 모니터링할 수 있습니다.



Work In Progress (WIP)

현재 동시에 진행 중인 작업의 수입니다. WIP 제한(Limit) 준수 여부를 확인하여, 과도한 멀티태스킹으로 인한 낭비를 줄이고 작업 흐름을 원활하게 합니다.



커스텀 차트 데이터 처리

GraphQL API로 Issue 데이터를 수집하고 Cycle Time을 계산하는 로직 구현

 query.graphql

GRAPHQL

```
# GitHub GraphQL API로 완료된 이슈 수집
query($owner: String!, $repo: String!) {
  repository(owner: $owner, name: $repo) {
    issues(first: 100, states: CLOSED) {
      nodes {
        title
        createdAt
        closedAt
        labels(first: 10) {
          nodes {
            name
          }
        }
      }
    }
  }
}
```

 calculate_metrics.js

JAVASCRIPT

```
// 수집된 데이터를 바탕으로 Cycle Time 계산
const issues = data.repository.issues.nodes;

// 각 이슈별 소요 시간(일 단위) 계산
const cycleTime = issues.map(issue => {
  const created = new Date(issue.createdAt);
  const closed = new Date(issue.closedAt);

  // 밀리초 -> 일 단위 변환
  return (closed - created) /
    (1000 * 60 * 60 * 24);
});

// 전체 평균 Cycle Time 도출
const avgCycleTime = cycleTime.reduce((a, b) => a + b)
  / cycleTime.length;

console.log(`Average Cycle Time: ${avgCycleTime.toFixed(1)} days`);
```




SECTION 11

효과적인 작업 분해

User Story와 Task 분해를 통해
복잡한 요구사항을 실행 가능한
단위로 관리하기

- ✓ User Story 작성법 (Who, What, Why)
- ✓ INVEST 원칙
- ✓ Task 분해 및 추정



User Story 작성법

사용자 관점에서 가치를 명확하게 정의하는 표준 템플릿과 예시



표준 템플릿 구조

As a [Role / 사용자 역할]

I want [Feature / 기능]

So that [Benefit / 혜택]

기술적인 구현 방식(How)이 아닌, **누가(Who)**, **무엇을(What)**, **왜(Why)** 원하는지에 초점을 맞춰 작성합니다. 이를 통해 개발자와 비즈니스 담당자 간의 명확한 소통이 가능해집니다.



작성 예시 (비밀번호 재설정)

As a 가입된 사용자

I want 비밀번호를 재설정 하기를

So that 계정을 다시 이용할 수 있도록

단순히 "비밀번호 재설정 기능 구현"이라고 적는 것보다, 사용자가 얻게 되는 가치(계정 복구)를 명시함으로써 해당 기능의 중요도와 목적을 팀 전체가 이해할 수 있습니다.



INVEST 원칙

좋은 User Story를 작성하기 위한 6가지 핵심 품질 기준

I Independent 독립적

각 User Story는 서로 의존하지 않아야 합니다. 의존성이 없어야 순서를 자유롭게 조정하고 개발할 수 있습니다.

N Negotiable 협상 가능

고정된 계약이 아닌, 협의의 대상이어야 합니다. 개발 중 더 나은 방법이 발견되면 내용을 조정할 수 있습니다.

V Valuable 가치 있음

사용자나 비즈니스에 실질적인 가치를 제공해야 합니다. 가치가 없는 작업은 백로그에 포함되지 않아야 합니다.

E Estimable 추정 가능

개발 팀이 작업의 크기와 난이도를 예측할 수 있어야 합니다. 명확하지 않다면 더 구체화해야 합니다.

S Small 작음

하나의 스프린트 내에 완료할 수 있을 만큼 작아야 합니다. 너무 크다면 더 작은 단위로 분할해야 합니다.

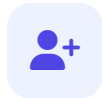
T Testable 테스트 가능

완료 여부를 판단할 수 있는 명확한 테스트 기준 (Acceptance Criteria)이 있어야 합니다.



Task 분해 예시

≡ Epic: 사용자 인증 시스템



Story: 사용자 회원가입

- ✓ 회원가입 UI 디자인
- ✓ 회원가입 폼 구현
- ✓ 이메일 검증 로직
- ✓ 비밀번호 암호화
- ✓ 가입 완료 이메일 발송



Story: 사용자 로그인

- ✓ 로그인 UI 구현
- ✓ JWT 토큰 생성
- ✓ 세션 관리
- ✓ "로그인 유지" 기능



Story: 비밀번호 재설정

- ✓ 재설정 요청 페이지
- ✓ 이메일 발송 로직
- ✓ 재설정 토큰 검증
- ✓ 새 비밀번호 저장



SECTION 12

Sprint Planning

성공적인 스프린트를 위한
목표 설정과 실행 계획 수립 프로세스

- 🎯 Sprint Goal 설정
- 📅 Capacity & Velocity 계산
- 📋 Task 분해 및 할당



| Sprint Planning 요약

성공적인 스프린트 시작을 위한 프로세스와 핵심 지표 계산법



표준 프로세스 (Timebox)

- Goal 설정 (30분): 스프린트 목표 합의
- Refinement (1시간): 백로그 상세화
- Capacity (30분): 가용 리소스 산정
- Tasking (1시간): 작업 분해 및 할당
- Commit (15분): 최종 팀 합의 및 시작



팀 Capacity 산정

- 공식: 팀원 수 × 근무일 × 일일 가용시간 × 이용률
- 가정: 회의, 이메일 등 제외한 이용률 80% 적용

$$5\text{명} \times 10\text{일} \times 6\text{h} \times 0.8 = \mathbf{240\text{시간}}$$



Velocity 기반 목표 설정

- '어제의 날씨' 원칙: 최근 3회 평균으로 예측
- 무리한 목표보다는 데이터 기반의 예측치 사용

$$(35\text{pts} + 42\text{pts} + 38\text{pts}) / 3 \approx \mathbf{38\ Points}$$



성공적인 플래닝 원칙

- 팀 합의: PO가 지시하는 것이 아닌 팀이 약속
- 현실성: 휴가, 공휴일, 교육 등 비가용 시간 제외
- 여유(Buffer): 예기치 못한 이슈 대응 시간 고려
- 준비 완료: DoR(Definition of Ready) 충족 확인



실습 과제

이번 주차에 수행해야 할 핵심 프로젝트 및 활동

01 GitHub Projects 설정



- ✓ 개인 프로젝트용 GitHub Project (v2) 생성
- ✓ 칸반 보드 구성 (Backlog, To Do, In Progress, Review, Done)
- ✓ 커스텀 필드 추가 (Priority, Story Points, Sprint) 및 자동화 설정

02 Issues & Milestones



- ✓ Issue 템플릿(Bug, Feature) 작성 및 10개 이상 이슈 생성
- ✓ Label 체계 구축 (Type, Priority, Status 등)
- ✓ Milestone 2개 생성(예: MVP, v1.1) 후 관련 Issue 할당

03 Sprint 실행



- ✓ 2주 Sprint 계획 수립 및 Sprint Goal 설정
- ✓ Sprint Backlog 구성 및 각 작업에 Story Points 할당
- ✓ Daily 진행상황 업데이트 및 Sprint Review 문서 초안 작성

04 메트릭 수집 및 분석



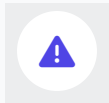
- ✓ 작업 완료 후 Cycle Time 측정
- ✓ Velocity 계산 및 팀 생산성 추세 분석
- ✓ Burndown Chart 생성 및 인사이트 도출 (개선점 찾기)



핵심 요약

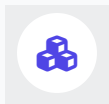
이번 주차의 주요 학습 내용 복습 및 다음 주차 예고

★ Key Takeaways



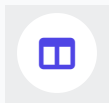
계획 없는 개발의 위험성

"Failing to plan is planning to fail" - 명확한 목표와 로드맵의 중요성



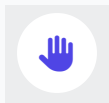
작업의 세분화 (Small Batch)

INVEST 원칙에 따라 작업을 작고 독립적인 단위로 분해



시각화 (Visualization)

칸반 보드와 차트를 통해 진행 상황과 병목을 투명하게 관리



WIP (Work In Progress) 제한

동시에 진행하는 작업 수를 제한하여 컨텍스트 스위칭 최소화



지속적 개선 (Retrospective)

정기적인 회고를 통해 프로세스를 점진적으로 개선



Best Practices

- ✓ 구체적이고 명확한 Issue 작성
- ✓ 일관된 Label 및 Milestone 사용
- ✓ 프로젝트 보드를 항상 최신 상태로 유지
- ✓ 데이터(메트릭) 기반의 의사결정

NEXT WEEK

GitHub 협업 심화

#Branch Strategy

#Code Review

#CI/CD Basics

Slide 48